Final Presentation

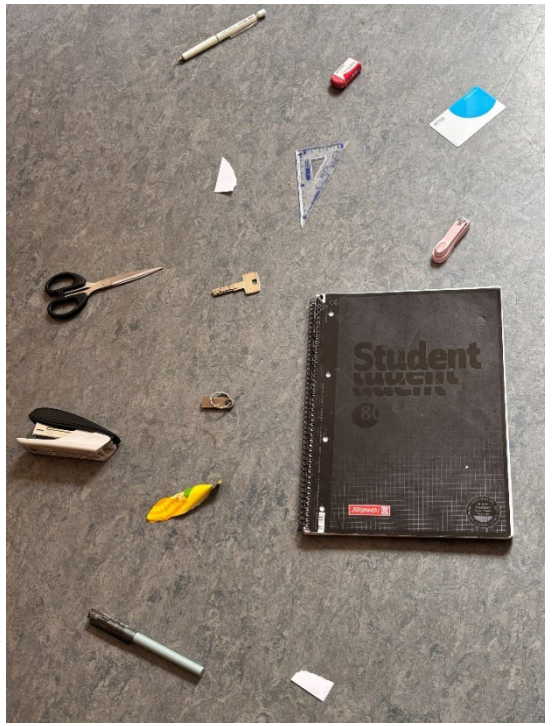
Projekt 02: Vision-based Dirt Detection for Cleaning Robot



Prepared by： Shufang Tao (3824652), Yuxin Song (3820779), Yuyang Ren (3762594)

Why do service robots need dirt detection?

- Indoor floor environments change constantly in real use.
- Dirt, litter, and small objects may appear unexpectedly.
- These objects can affect cleaning quality, navigation safety, and user experience.
- Vision-based dirt detection helps the robot understand the actual floor condition.
- This enables more adaptive and intelligent cleaning behavior.



Key idea

Detection turns floor images into cleaning decisions.

robot view — floor condition — cleaning action

See
Decide
Clean

Project goals

Core objective

- Build a YOLO-based floor object detection system for cleaning robot vision.
- Compare YOLO architectures and select the most suitable framework for optimization.

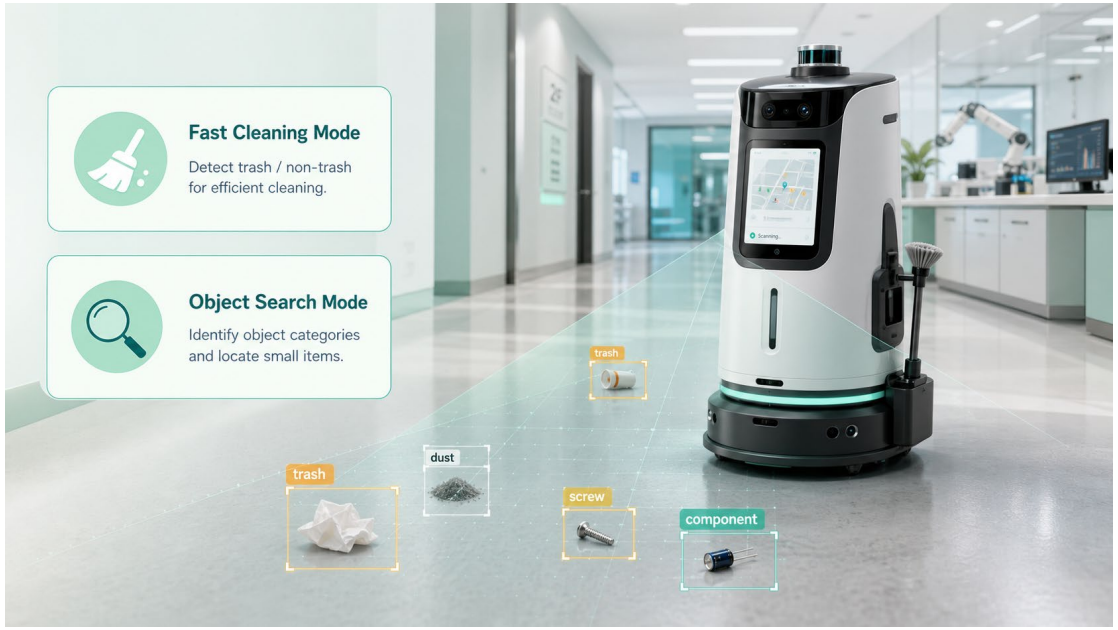
Product-oriented detection modes

Support two detection modes for different product needs:

Binary mode — Fast trash / non-trash detection for efficient cleaning.

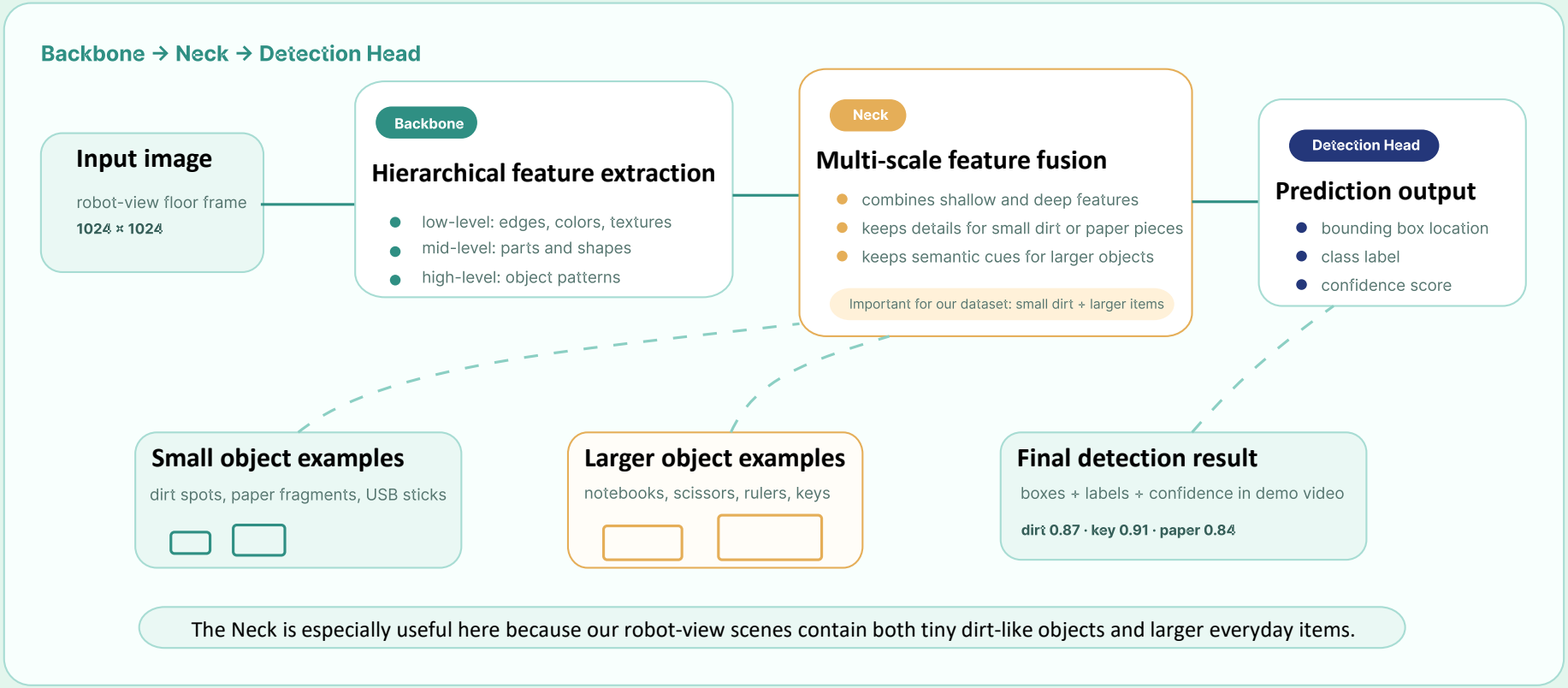
Multi-class mode — Recognize object categories and mark their positions.

For extra user needs: finding a small lost item in a corner.....



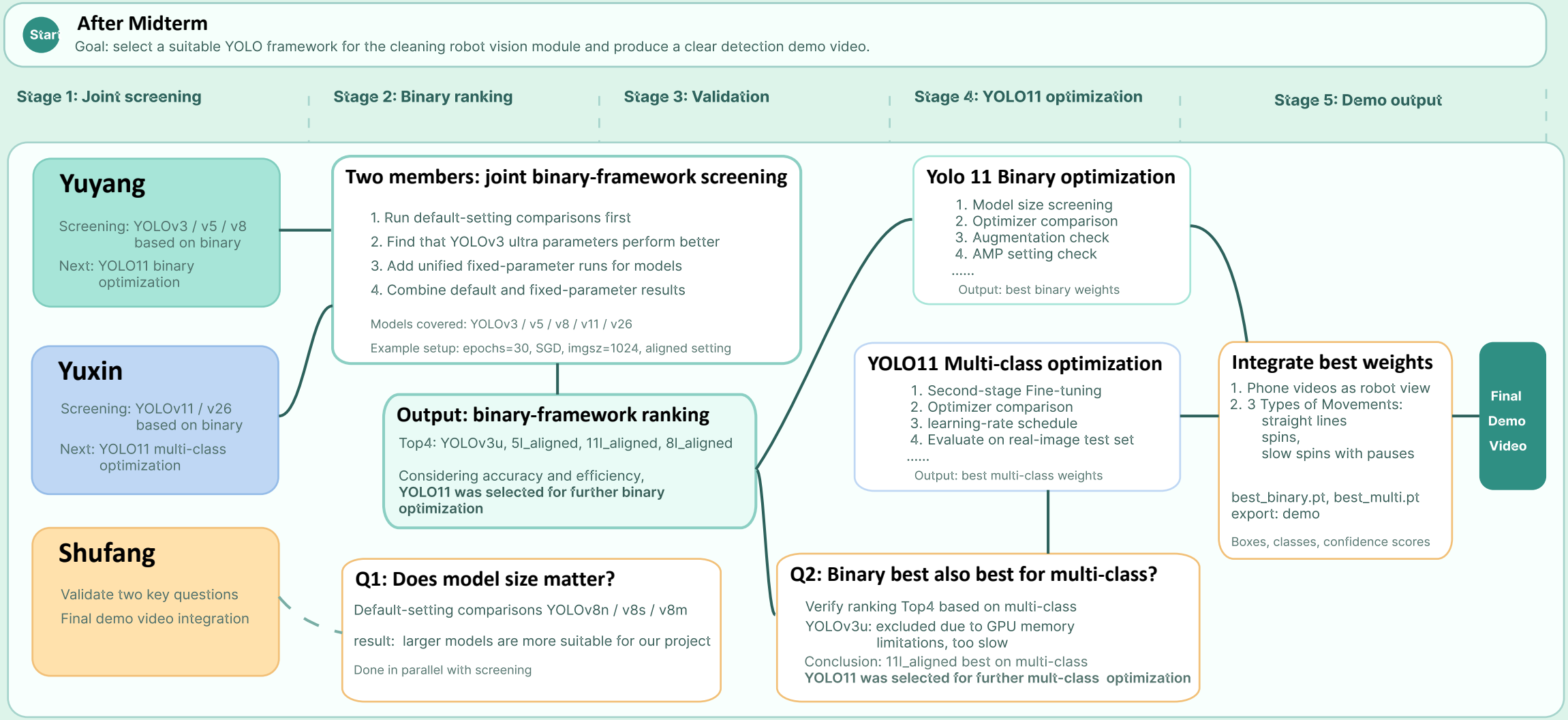
YOLO architecture for object detection

A high-level view of how YOLO converts a robot-view image into boxes, classes, and confidence scores.



Cleaning Robot Vision Module: Team Workflow and Model Selection

produce the model ranking; the ranking then supports optimization, and demo integration.



Q1: Does model size matter?

YOLOv8 Nano, Small, and Medium were compared to test whether larger model capacity improves detection results.

mAP50-95 increases as model size grows **larger scale → stronger detection**

Best mAP50-95 result for each YOLOv8 scale within the 30-epoch experiment.



Best-epoch metrics

Model	Precision	Recall	mAP50	mAP50-95
YOLOv8n	0.88462	0.81762	0.87501	0.71009
YOLOv8s	0.94336	0.91745	0.94842	0.79568
YOLOv8m	0.95863	0.93536	0.95049	0.81288

Conclusion S / M

Within the tested YOLOv8 scales, larger models achieved better multi-class detection results.

This suggests that relatively larger models are more suitable for our project than Nano scale.

Q2: Binary-further-optimization baseline also best for multi-class?

YOLOv5LU, YOLOv8L, and YOLOv11L were validated under aligned multi-class training settings.

Experimental premise

Input from teammates
Top 4 from binary-framework ranking

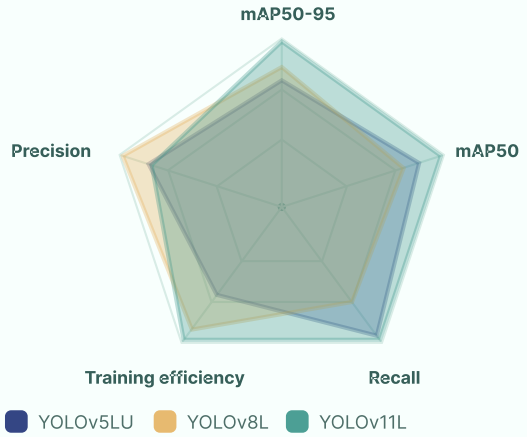
Validation candidates
YOLOv5LU: completed
YOLOv8L and YOLOv11L: completed
YOLOv3u: excluded due to GPU memory

Aligned setting

```
imgsz=1024 batch=2
epochs=30 optimizer=SGD
lr0=0.01 lrf=0.01
mosaic=1.0 mixup=0.0
erasing=0.0 auto_augment=none
amp=False
```

Comprehensive comparison

Best results selected by mAP50-95.



Decision signal
11L wins on Recall, mAP50, mAP50-95, and time.
8L has the highest Precision; 5LU is close but slower.

Best-epoch metrics

Metric	5LU	8L	11L
Epoch	30	29	29
Precision	0.96439	0.97198	0.96432
Recall	0.93563	0.92776	0.93674
mAP50	0.96360	0.96152	0.96715
mAP50-95	0.82001	0.82417	0.83170
Time	132.0	97.4	91.5

Selected: YOLOv11 for further Multi-class optimization

The margin is moderate, but 11L is the most consistent candidate across key multi-class metrics and training time.

Demo video pipeline

Phone videos were used to simulate a robot-view perspective and evaluate how motion affects visual detection quality.

Pipeline

Record
phone as robot view

→

Test Motion
different movement styles

→

Run YOLO
best trained weights

→

Export Demo
boxes, classes, confidence

Motion strategies tested

Method 1

Straight fast movement

Stable view and lower motion blur.

Good for unobstructed fast-cleaning paths.

Method 2

Larger rotation

Large blur and unstable object boxes.

fast multi-angle approach didn't improve recognition

Method 3

Slow spin with pauses

Object stays longer in the camera view.

Good for obstacle areas and object checking.

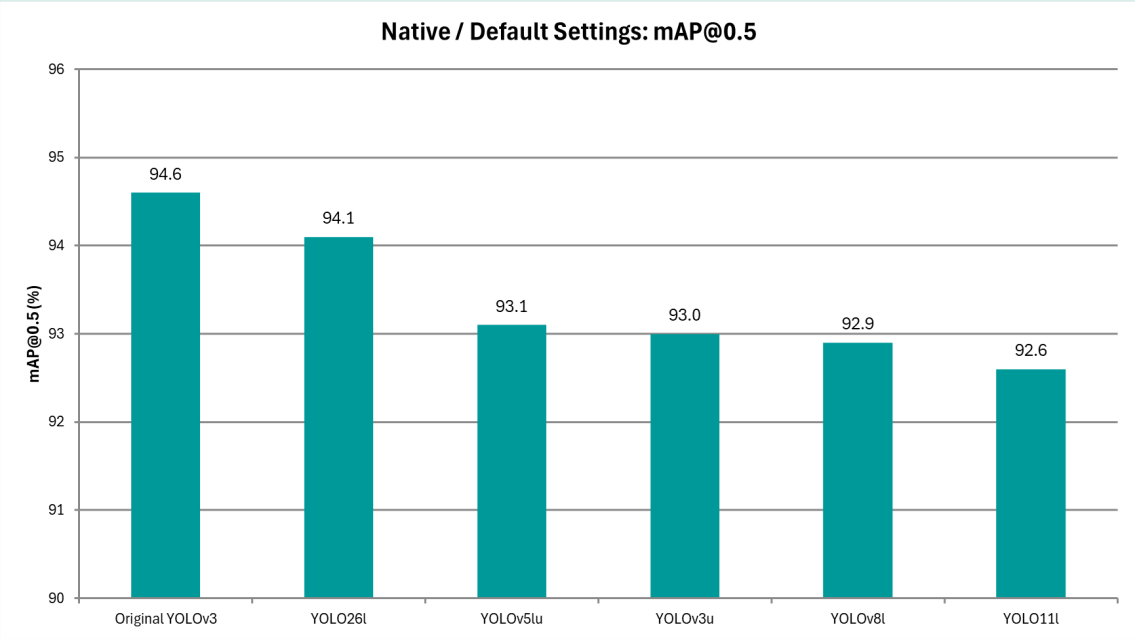
Observation for future robot behavior

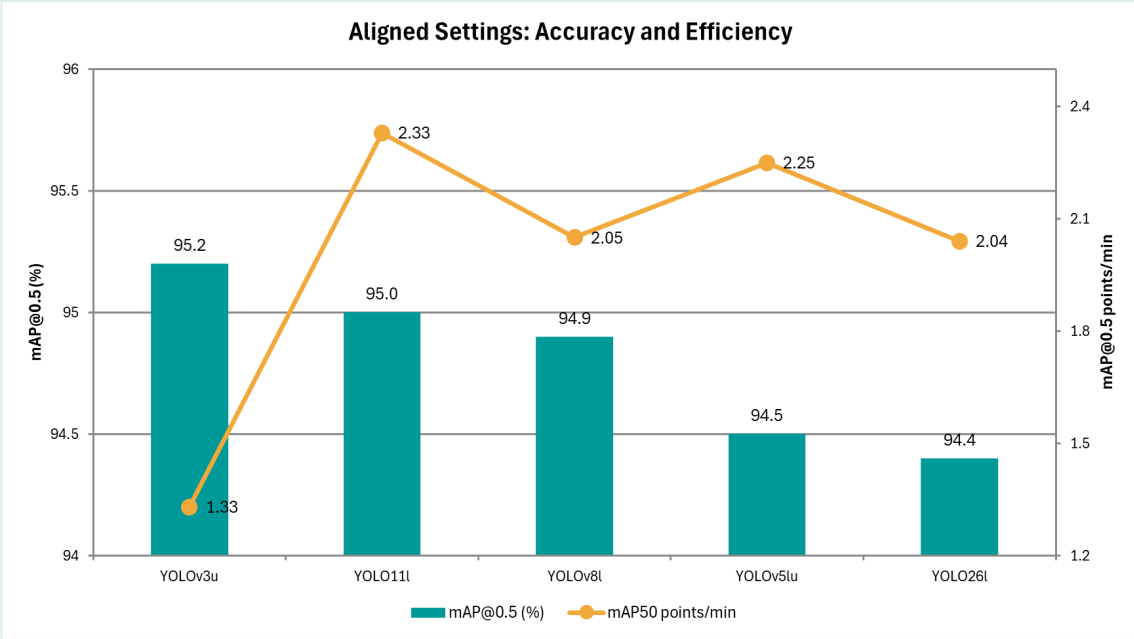
For unobstructed areas, straight movement can support faster cleaning.
For obstacle or object-rich areas, slow rotation with short pauses is more detection-friendly.

Presentation note

The final generated demo video will be shown at the end of the presentation.

YOLO Version Screening





Comparison	Evidence	Decision
Native / default	Original YOLOv3 remained strong at 94.6 mAP@0.5.	Configuration differences affected ranking.
Aligned setting	YOLO11l reached 95.0 mAP@0.5 with far less time than YOLOv3u.	Use YOLO11 as the main family.

Takeaway: YOLO11 family provided near-best binary accuracy with a better training-time trade-off, so it became the basis for later optimization.

YOLO11 Binary Optimization

Dataset scale-up for later optimization

8x larger

Small pilot	1,156 train/val images | 11,588 boxes
Full dataset	9,248 train/val images | 92,552 boxes

Model Size Screening

Model	mAP50	mAP50-95	Time	Role
YOLO11n	0.929	0.785	21.5 min	too weak
YOLO11s	0.956	0.834	34.3 min	screening
YOLO11m	0.962	0.854	69.3 min	candidate

Screen with YOLO11s -> apply stable setting to YOLO11m.

YOLO11m is more accurate, but about 2x slower than YOLO11s.

Training Settings

Optimizer **Use SGD**

SGD > Default (AdamW)

Augmentation **Keep default**

extra changes: little gain

AMP(Automatic Mixed Precision)

amp=False much slower **Keep enabled**

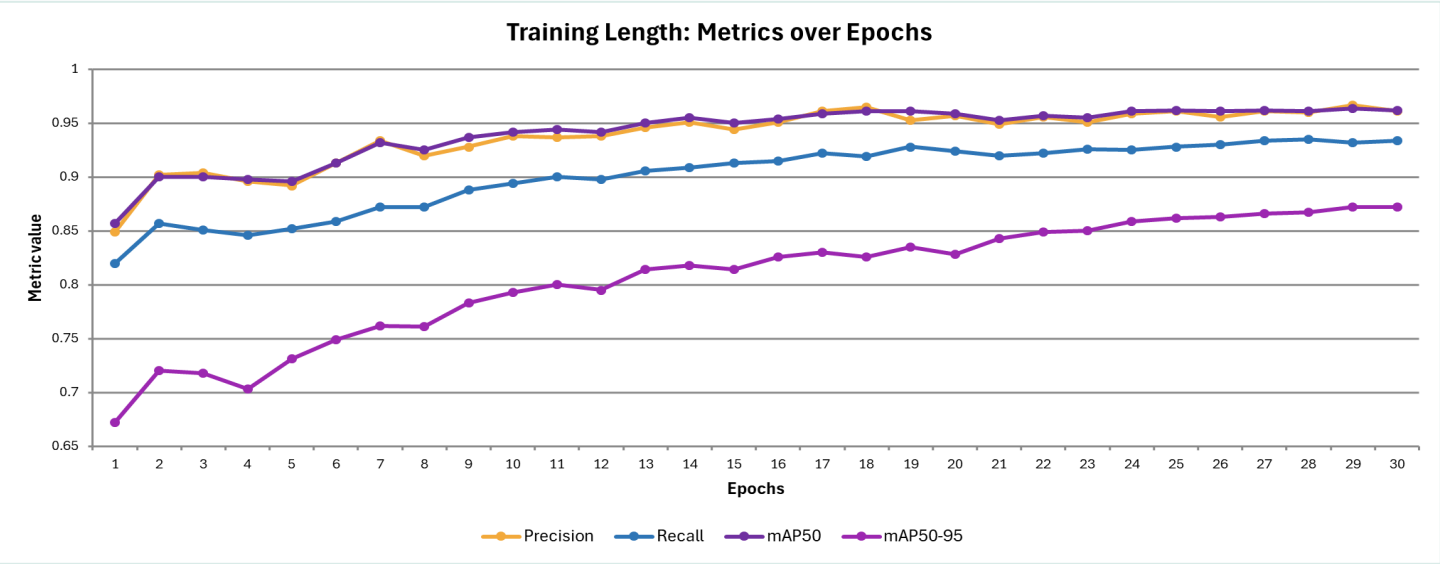
Cosine Learning Rate **Keep default**

same as linear learning-rate schedule

Current Choice

- YOLO11s: parameter screening
YOLO11m: high-capacity candidate
- optimizer: SGD
- augmentation: default
- AMP: enabled
- learning-rate schedule: default

Resolution and Training Strategy



Training length decision

30 epochs improved recall and localization quality.

15 epochs stayed as the faster screening setting.

Final runs should use longer training; parameter search remains shorter.

Resolution comparison

Setting	imgsz	Epochs	Recall	mAP50	mAP50-95
YOLO11s fast	1024	15	0.923	0.959	0.844
YOLO11s longer	1024	30	0.934	0.962	0.872
YOLO11s high-res	1280	15	0.944	0.977	0.876

Increasing input size from 1024 to 1280 produced a clear gain for small dirt regions.

Practical constraints and rect=True

- **GPU memory limited broad YOLO11m + 1280 search.**
- **YOLO11s + 1280 was compared with YOLO11m + 1024 under practical cost constraints, suggesting that resolution was more useful than simply increasing model size under our constraints.**

rect=True: faster, but disables mosaic and slightly lowers metrics.

Takeaway: longer training improved the detector, higher input resolution added a clear gain, and rect=True offered speed at a small accuracy cost.

Final Binary Candidates: Validation vs Real Test

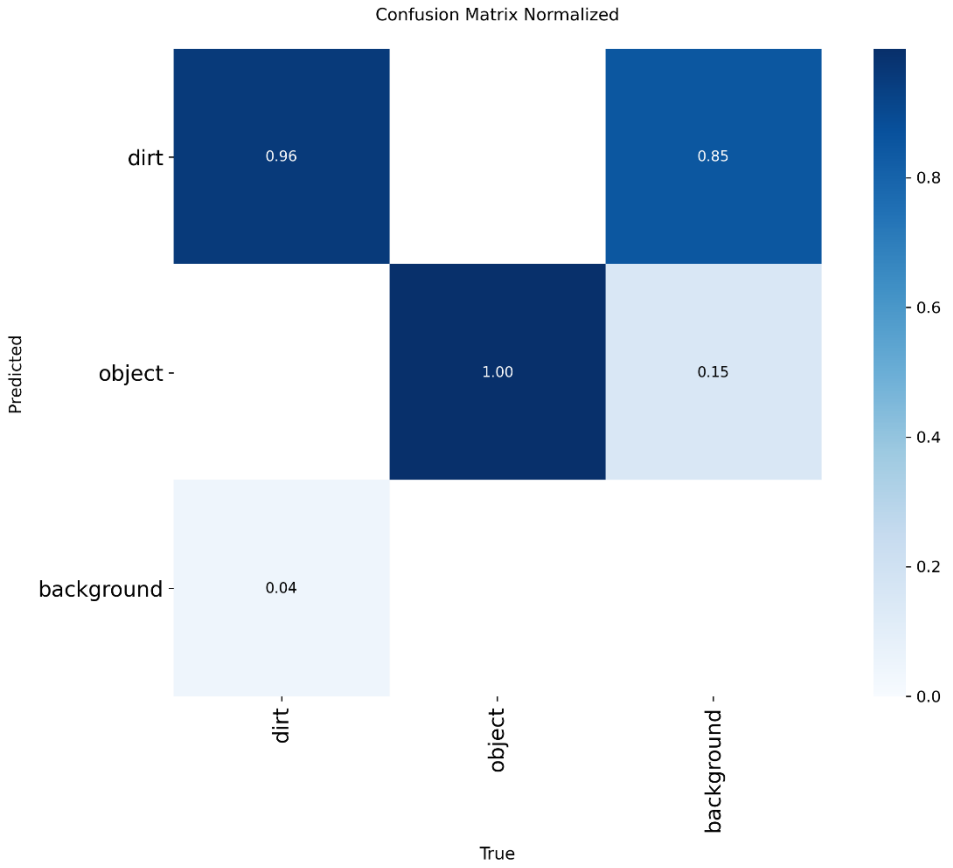
Final Training and Validation

imgsz=1280 · epochs=30 · optimizer=SGD · batch=8/4

YOLO11m used batch 4 because of GPU memory pressure.

Model	Setting	P	R	mAP50	mAP50-95
YOLO11s	1280 · 30e · b8	0.972	0.951	0.979	0.906
YOLO11m	1280 · 30e · b4	**0.974**	**0.962**	**0.983**	**0.923**

Validation-selected model: YOLO11m



Final Binary Candidates: Validation vs Real Test

Final Training and Validation

imgsz=1280 · epochs=30 · optimizer=SGD · batch=8/4

YOLO11m used batch 4 because of GPU memory pressure.

Model	Setting	P	R	mAP50	mAP50-95
YOLO11s	1280 · 30e · b8	0.972	0.951	0.979	0.906
YOLO11m	1280 · 30e · b4	**0.974**	**0.962**	**0.983**	**0.923**

Validation-selected model: YOLO11m

Drop on real images

mAP50

YOLO11s -0.173

YOLO11m -0.205

mAP50-95

s: -0.344 m: -0.391

Validation scores did not transfer directly.

Testing with real images

Model	P	R	mAP50	mAP50-95	Infer
YOLO11s	0.781	**0.772**	**0.806**	**0.562**	**10.0**
YOLO11m	**0.835**	0.704	0.778	0.532	21.4

YOLO11m kept higher precision, but YOLO11s achieved better recall, mAP, and speed.

Real test favored YOLO11s

Inference shown in ms/img.

Validation performance selected YOLO11m, but real-test behavior favored YOLO11s. Detailed reasons for this real-domain drop are discussed in the following section.

The binary DirtNet task became a 10-class detector

10 classes

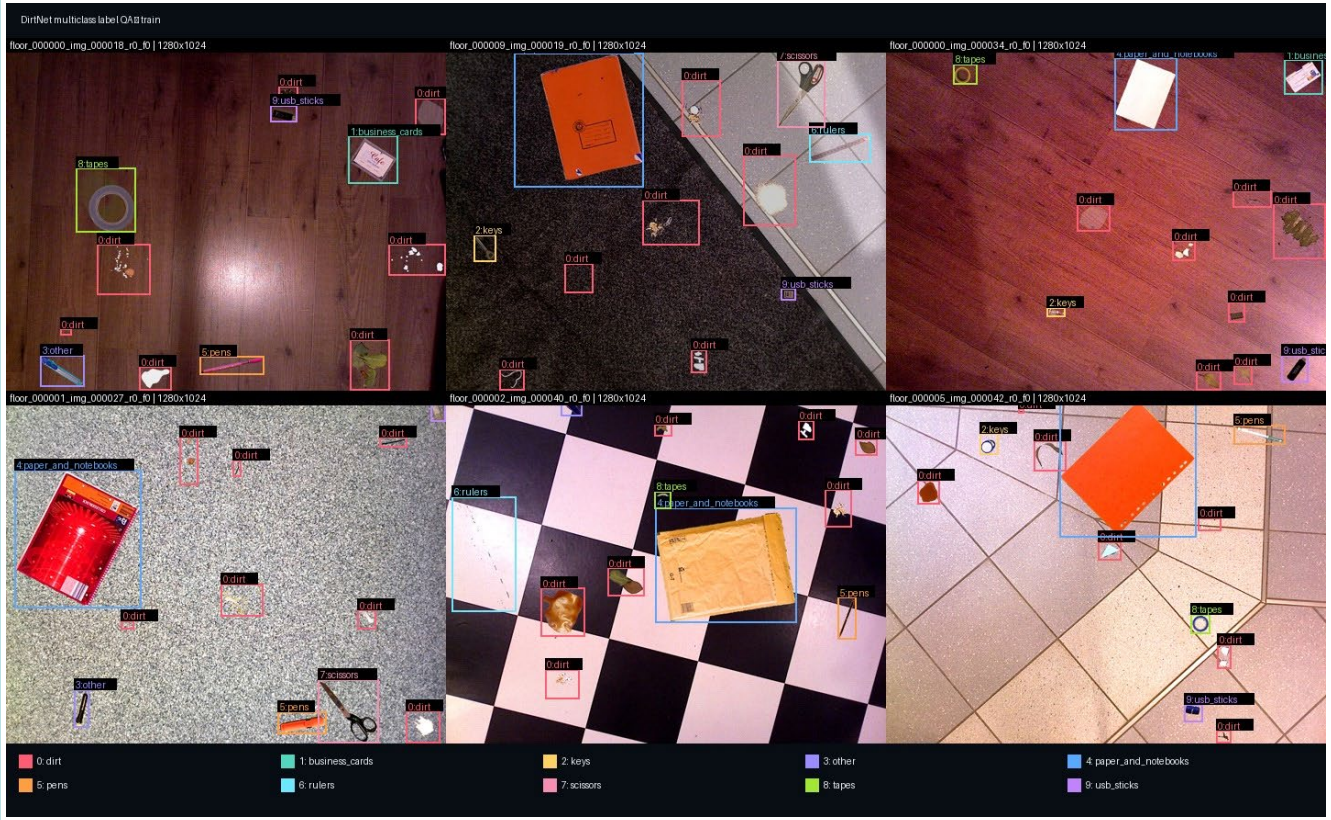
dirt + 9 office-object categories

Dataset

860 training images · 8,626 boxes
296 validation images · 2,962 boxes
80 held-out real images · 644 boxes

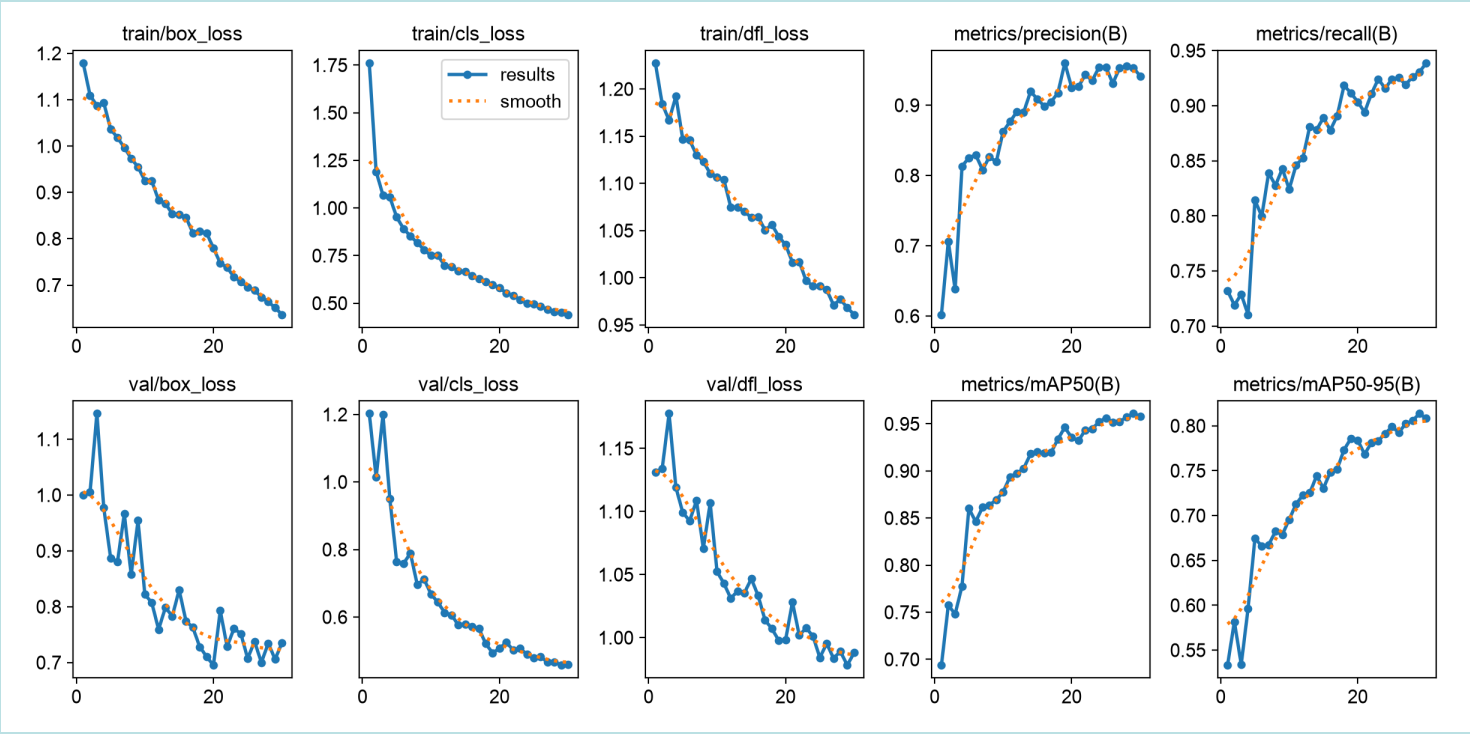
Controlled baseline

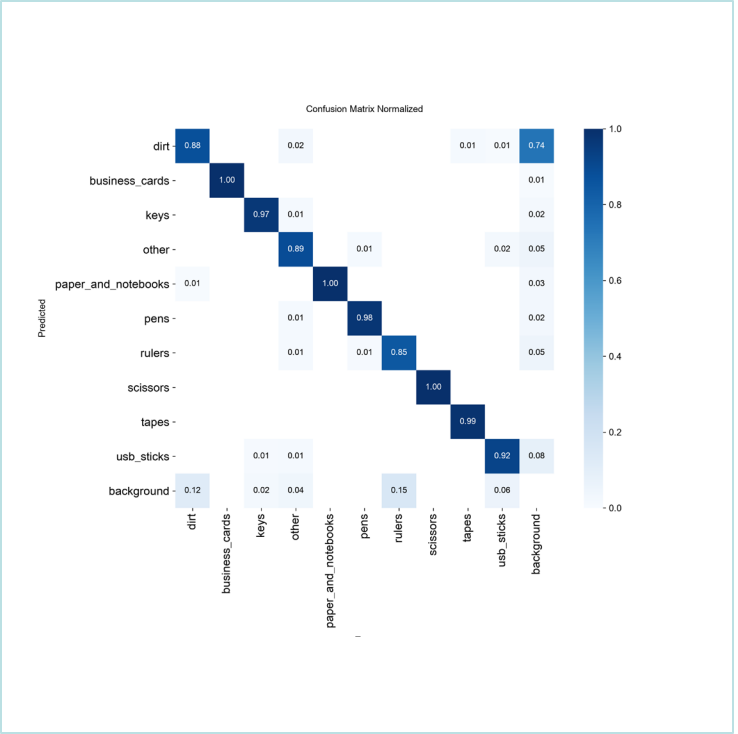
YOLO11M · 1024 px · batch 2
30 epochs on synthetic data
Independent real-world evaluation



business cards · keys · other · paper/notebooks · pens · rulers · scissors · tapes · USB sticks

Synthetic validation confirmed a working baseline





0.953	0.930	0.961	0.814
Precision	Recall	mAP@0.5	mAP@0.5–0.95

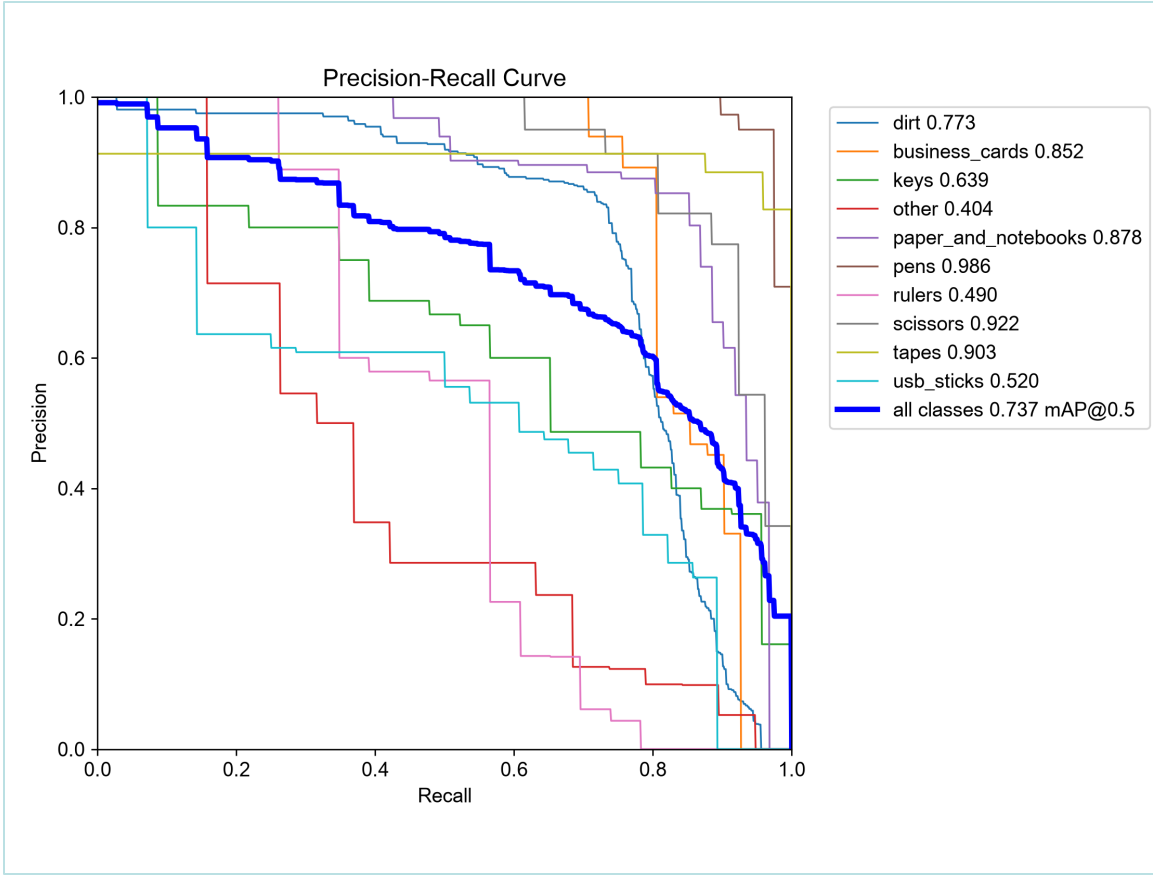
Meaning: the 10-class labels, output head and training pipeline function correctly.

Real images exposed a clear domain gap

	Synthetic validation	Independent real test
Precision	**0.953**	**0.665**
Recall	**0.930**	**0.776**
mAP@0.5	**0.961**	**0.737**
mAP@0.5–0.95	**0.814**	**0.554**

mAP@0.5 dropped by 0.224

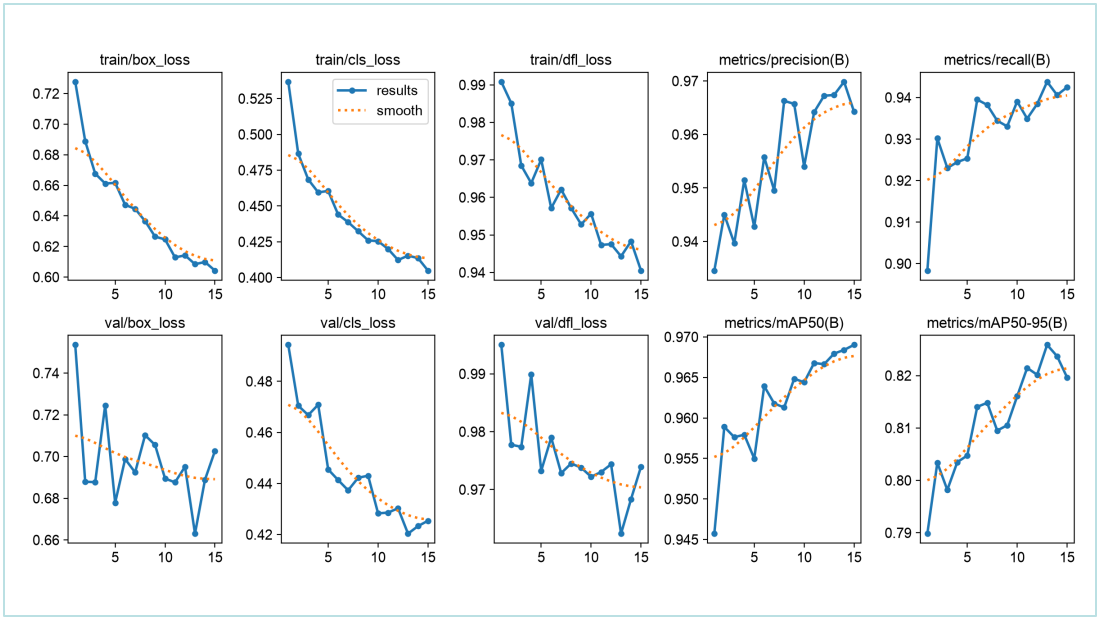
Strong: pens, scissors, paper/notebooks
Weak: other, rulers, USB sticks
Reflections, shadows and floor texture increase false positives



The gap affects both classification and box localization.

Low-rate fine-tuning did not improve real transfer

Modified parameter	Why it was changed
Baseline best.pt	Refine an already converged solution
AdamW	Adaptive steps + decoupled weight decay
$lr_0 = 1\times10^{-4}$	Smaller updates preserve learned features
Cosine decay → 1×10^{-5}	Smooth convergence near the optimum
Mosaic disabled	Stabilize layout and localization
15 epochs · patience 5	Bounded refinement; stop early



Synthetic mAP@0.5

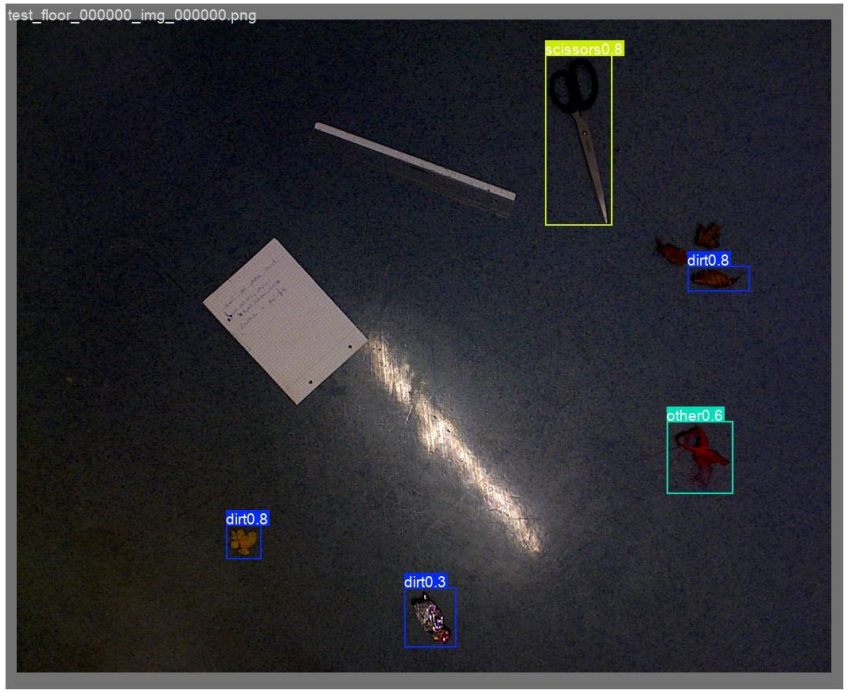
0.961 → 0.968

Real mAP@0.5

0.737 → 0.726

Further optimization fitted the synthetic domain more closely, but did not close the real-world gap.

Further gains require real-world data



Correct detections coexist with missed objects and reflective-floor ambiguity.

Critical discussion & lessons learned

High synthetic scores do not guarantee real-world performance.
More epochs and lower learning rates cannot solve domain mismatch.
Class balance, label definitions and data distribution dominate the remaining error.
The broad “other” class introduces substantial ambiguity.

Outlook and future work

Collect separate real training and validation data.
Add weak classes and difficult negative samples.
Model reflections, shadows, scratches and low light.
Redefine “other” and tune confidence for robot priorities.

A functional multiclass baseline is established; real-domain adaptation is the next step.